

```
Particle swarm optimisation - x  Untitled5.ipynb - Colab  Upload files - amrutanand/E...
colabresearch.google.com/drive/1_toLueCPJOy0WYhNjWGND5rdFvpQaMK#scrollTo=NatMYLduRCh

Untitled5.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

import numpy as np
from PIL import Image
import random

image = Image.open("../Screenshot_2026-10-05_111909.png").convert("L")
image = np.array(image)

def fitness(threshold):
    background = image[image < threshold]
    foreground = image[image >= threshold]

    if len(background) == 0 or len(foreground) == 0:
        return 0

    w1 = len(background) / image.size
    w2 = len(foreground) / image.size

    m1 = np.mean(background)
    m2 = np.mean(foreground)

    variance = w1 * w2 * (m1 - m2) ** 2

    return variance

num_particles = 10
iterations = 20

w = 0.5
c1 = 1.5
c2 = 1.5

positions = [random.randint(0, 255) for _ in range(num_particles)]
velocities = [random.uniform(-5, 5) for _ in range(num_particles)]

pbest = positions.copy()
pbest_value = [fitness(x) for x in positions]

gbest_index = np.argmax(pbest_value)
gbest = pbest[gbest_index]
```

```
Particle swarm optimisation - x  Untitled5.ipynb - Colab  Upload files - amrutanand/E...
colabresearch.google.com/drive/1_toLueCPJOy0WYhNjWGND5rdFvpQaMK#scrollTo=NatMYLduRCh

Untitled5.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

for _ in range(iterations):
    for i in range(num_particles):
        r1 = random.random()
        r2 = random.random()

        velocities[i] = (
            w * velocities[i]
            + c1 * r1 * (pbest[i] - positions[i])
            + c2 * r2 * (gbest - positions[i])
        )

        positions[i] += velocities[i]
        positions[i] = max(0, min(255, positions[i]))

        current_fitness = fitness(positions[i])

        if current_fitness > pbest_value[i]:
            pbest[i] = positions[i]
            pbest_value[i] = current_fitness

        best_index = np.argmax(pbest_value)

        if pbest_value[best_index] > fitness(gbest):
            gbest = pbest[best_index]

    threshold = int(gbest)
    segmented = np.where(image < threshold, 0, 255)
    Image.fromarray(segmented.astype(np.uint8)).save("segmented_image.jpg")

    print("Optimal Threshold:", threshold)
    print("Segmented image saved as segmented_image.jpg")

Optimal Threshold: 150
Segmented image saved as segmented_image.jpg
```